

4

Le langage SQL

Introduction

SQL (Structured Query Language) qu'on peut traduire en français "langage de requêtes structurées" est le langage standard utilisé le plus couramment aujourd'hui pour créer, modifier, lire, mettre à jour, ... des bases données relationnelles.

Il existe 5 type de commandes SQL : DDL (Data Definition Language), DML (Data Manipulation Language), DCL (Data Control Language), TCL (Transaction Control Language), et DQL (Data Query Language).

4.1 Le langage de définition des données

C'est le sous-ensemble de SQL qui permet de pour manipuler les structures de données d'une base de données, et non les données elles-mêmes.

Il permet :

1. de définir le domaine des données, c'est-à-dire l'ensemble des valeurs que peut prendre une donnée : nombre, chaîne de caractères, date, booléen.
2. de regrouper les données ayant un lien conceptuel au sein d'une même entité.
3. de définir les liens entre plusieurs entités de nature différente.
4. d'ajouter des contraintes de valeur sur les données.

On distingue typiquement trois types de commandes SQL de définition de données : Ces commandes peuvent porter sur les struc-

Commande	Description
CREATE	création d'une structure de données
ALTER	modification d'une structure de données
DROP	suppression d'une structure de données

tures de données de type suivantes : DATABASE (base de données), TABLE table, INDEX (indice), VIEW (table virtuelle), SEQUENCE (suite de nombres), SYNONYM (synonyme) et USER (utilisateur).

4.1.1 La commande CREATE

Pour créer une base de données la commande est :

```
1 CREATE DATABASE mabase;
```

Alors que pour une table on devra écrire

```
1 CREATE TABLE nom_table (
2     colone1 type,
3     colone2 type,
4     colone3 type,
5     ....
6 );
```

Chaque champs d'une table possède un type : CHAR, VARCHAR, INTEGER, NUMBER, DECIMAL, FLOAT, DOUBLE, DATE, TIME, TIMESTAMP, etc.

Ainsi, la création de la table serais :

```
1 CREATE TABLE Personne (
2     ID integer,
3     Nom varchar(25),
4     Prenom varchar(50),
```

5
6

```
Adresse varchar(255)  
);
```



Lors de la création d'une table on doit définir l'identifiant avec PRIMARY KEY. Il est aussi possible qu'une clé étrangère soit présente dans une table dans ce cas on devra le marquer par FOREIGN KEY.

1
2
3
4
5
6
7

```
CREATE TABLE Commande (  
    IDCommande int NOT NULL,  
    Montant int NOT NULL,  
    IDPersonne int,  
    PRIMARY KEY (IDCommande),  
    FOREIGN KEY (IDPersonne) REFERENCES Personne(ID)  
);
```

Le langage SQL est normalisé, c'est-à-dire qu'un organisme s'est chargé d'établir la bonne façon de l'utiliser afin que le langage soit semblable d'un SGBD à l'autre.



Les différents SGBDR qui l'utilisent n'ont cependant pas suivi toutes les normes du standard SQL, ce qui fait que la syntaxe sera légèrement différente de l'un à l'autre. Les fonctionnalités supportées dépendent également du SGBDR.

Voici un exemple de requête SQL qui sera différente entre les SGBD :

Exemple :
MySQL

1
2
3
4

```
CREATE TABLE 'etudiants' (  
    'id' int(11) NOT NULL AUTO_INCREMENT,  
    'nomfamille' varchar(50) COLLATE utf8_unicode_ci,  
    'prenom' varchar(50) COLLATE utf8_unicode_ci,
```

```

5      'da' varchar(6) COLLATE utf8_unicode_ci,
6      PRIMARY KEY('id')
7 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci
  ;

```

SQLite

```

1 CREATE TABLE IF NOT EXISTS 'etudiants' (
2     'id' INTEGER PRIMARY KEY AUTOINCREMENT,
3     'nomfamille' TEXT,
4     'prenom' TEXT,
5     'da' TEXT);

```

MS SQL

```

1 CREATE TABLE [dbo].[etudiants] (
2     [id] INT IDENTITY(1, 1) NOT NULL,
3     [nomfamille] NVARCHAR(50),
4     [prenom] NVARCHAR(50),
5     [da] NVARCHAR(6)
6 );

```

4.1.2 La commande ALTER

Elle permet de modifier un champs d'une table, d'ajouter un nouveau champs ou d'en supprimer un.

```

1 ALTER TABLE Personne ADD (Telephone integer);
2 ALTER TABLE Personne MODIFY (Nom varchar(10));
3 ALTER TABLE Personne DROP Nom ;

```

4.1.3 La commande DROP

```

1 DROP DATABASE nombase;
2 DROP TABLE nomtable;

```